Enriching the Semantics of Information Flow Tracking with Source-Level Memory Allocation Event Logging

Sanoop Mallissery, *Member, IEEE,*, Kun-Yi Chiang, and Yu-Sung Wu†, *Member, IEEE*
Department of Computer Science, *National Yang Ming Chiao Tung University, Hsinchu, Taiwan*
sanoopmallissery@gmail.com, kkz0xb17@gmail.com, ysw@nycu.edu.tw

***Abstract*—A common practice for conducting code analysis is injecting probes (external instrumentation of code) into the application, which requires exploration. Such explorations comprehend monitoring or debugging the code, finding bugs, performance analysis, and memory profiling. Code instrumentation can be performed statically/dynamically, and the code analysis tools focus on a specific task by using hard-coded commands for instrumentation. Here, we have devised SQUIRREL, a configurable static code instrumentation tool, which enriches the semantics of information flow representation at its source-level with detailed logging of each memory allocation event. It intrigues the end users/systems administrators to have precise insight into the code information flow, which contributes to securing the code from zero-day vulnerabilities or applications-specific logic loopholes. We evaluate SQUIRREL with various real-world programs and generate information flow that realizes source-level online *dynamic information flow tracking* (DIFT) and discusses the efficiency of SQUIRREL concerning performance, memory, and runtime profiling with existing profiling tools.**

***Index Terms*—code instrumentation, code analysis, dynamic information flow tracking, code sanitization**

I. Introduction

The elegance, meticulousness, and structuring of C programming language coding postulate it as a base language for developing dozens of best applications [1]. Indeed, it has played a crucial role in developing various Operating Systems [2]. However, it is noteworthy that the efficacy of all the applications concerning security against malware, ransomware, and other intrusions is questionable. If an application fails to execute as demanded regarding the security of the data it handles, the adverse implications are worse than ever. The exploits may lead to a series of incidents and impact the dependencies between numerous services provided by the applications. For example, by finding code vulnerabilities, striking it with zero-day attacks induce data stealing, file corruption, device controlling, malware, trojans, and worms [3]. Many high-profile technology companies were affected by such zero-day attacks due to the vulnerabilities in the code [4]. Wherefore, developing a safety-critical code is critical.

To develop a safety-critical application [5], we must follow (1) code control-flow integrity (CFI) [6], [7], which restricts untrusted goto, jumps, and direct or indirect recursions, (2) the fixed upper boundness on loops, (3) the defined scope of function calls, and to (4) restrict dynamic memory allocation (heap) after initialization. Therefore, to ensure the source code's safety, code execution should guarantee control flow, memory, and stack safety security [8]. It is necessary to ensure that code execution does not invoke a jump instruction to an unknown memory region [9]. Instead, it should be in its code segment with appropriate instructions to execute. There should not be any random access to the classes of memory regions except its allowed code segment (static or stack frames) and the explicitly allocated heap memory [10], [11]. Likewise, we must preserve the runtime stack throughout the scope of a function call. However, for large real-world applications, these kinds of restrictions may not work. There may be lines of code that are elusive to determine whether any regions of the code become vulnerable [12], [13]. We can utilize tools such as Valgrind-massif/memcheck[1], heaptrack/stackusage[2,3], and gperftools[4] to check the memory allocation statistics, memory usage, leaks, and bugs related to memory leaks and thus to ensure the safety of the original and instrumented source code/ binaries.

Instrumented source code/ binaries will have injected probes (external code instrumentation statically/dynamically) at critical code portions of the original code to perform a particular task. For example, monitor the information flow of the program at runtime. Here, we devised a configurable static instrumentation tool SQUIRREL (Static Quench to Unified Information Retrieval)* [14] to perform information flow tracking with source-level memory allocation event logging. The shadow tail of the SQUIRREL stealthily follows all the program's variables and functions/methods to render the precise sequential/ control, conditional, and iterative information flow. SQUIRREL fetches all the variable and function/method names at runtime by tracing the *Definition-Use* (*DU*) *chain* [15] and generating the information flow with memory allocation event logging. Hence, SQUIRREL makes the information flow human-readable for the end users/system administrators, essential for all kinds of applications, specifically the ones related to security forensics or malware analysis. Overall, SQUIRREL offers a combination of the following capabilities. (1) Instrument at all non-trivial source code locations (statically) where there is a declaration, initialization, and invocation (*DU Chain*). (2) Generate the program information flow with enriched semantics representation. (3) Memory event allocation logging for deepened and elevated information flow tracking. (4) Improving the readability of the information flow for the end users/system administrators.

A. Scope of SQUIRREL Source Code Instrumentation

SQUIRREL instruments the LLVM intermediate representation (IR)[5] code and enriches the semantics of the information flow tracking with source-level memory allocation events logging. We have omitted the discussion related to other programming languages (except C, and LLVM IR), which are part of the experimented applications. No changes have been made in the application's source code besides instrumenting the LLVM IR to acquire the scope of variables and functions. Therefore the original application source code remains intact without any manipulation. We use LLVM and WLLVM[6] and strictly follow LLVM debugging to make it interact well with the source code analysis envisaged. In particular, it debugs information to read the program's source-level state accurately, and WLLVM leverages it to aid with debugging and sanity checks. Also, it is equally important to successfully generate the processed files based on the error-free conversion to LLVM IR.

†Corresponding author.

*This work details the SQUIRREL instrumentation mechanism in our paper titled "**Pervasive Micro Information Flow Tracking**", *IEEE Transactions on Dependable and Secure Computing*, DOI: 10.1109/TDSC.2023.3238547

[1] https://valgrind.org/docs/ [2] https://github.com/KDE/heaptrack
[3] https://github.com/d99kris/stackusage [4] https://github.com/gperftools/
[5] https://llvm.org/docs/ [6] https://github.com/travitch/whole-program-llvm

II. Related Works

We can administer program analysis via dynamic/static [16], [17], and both techniques conceal some external code in their source or binary and are called instrumentation. However, the instrumentation can be performed at runtime (dynamic/compiler-based) [18]–[23]), or we can instrument the external code directly to the source code or rewrite the application binary before the execution (static [24]) of binary or the program. Researchers perform this based on the need for applications they are looking for or finding a suitable parametrization depending on the demanding task, such as application monitoring, malware analysis, or security forensics [25], [26]. In fact, there should be a trade-off among the performance, precision, scalability, speed, effectiveness, and fault tolerance of the instrumented binary or the source code. Leveraging sparse analysis with value-flow construction and pointer analysis is one another way to perform interprocedural source code analysis [27], [28]. Therefore, choosing the type of instrumentation is skeptical since both dynamic/static possess mixed ways of concern [29]. However, if the user desires to change the semantics of the instrumented code, the alternatives are minimal in dynamic instrumentation.

For example, tools such as TRITON[7], DynamoRIO[8], PIN [18], DynaPyt [30], and Valgrind [19] carry out dynamic binary instrumentation or analysis. However, the performance overhead associated with all these tools are high, which is crucial for critical applications such as security forensics and malware analysis. Valgrind is one of the best instrumentation frameworks for building dynamic analysis tools, and it also lodges memory management and bug finding. Moreover, Valgrind-massif helps examine the binary's memory (heap/stack) flaws. This interests us, and we use Valgrind, heaptrack/stackusage and gperftools to do memory profiling and help us deep dive into SQUIRREL instrumented binaries memory flaws.

Our instrumentation tool SQUIRREL does not do any memory management, bug finding, or memory profiling; instead, we try to advocate the end users/system administrators with detailed information flow of the program during its execution and provide a mapping of memory addresses to the original variables names in the source code. Furthermore, we intend to have a configurable tool that could be utilized based on the user's need and extended by incorporating the changes at the high-level language. Moreover, the instrumented code should not change the behavior of the original program and data/control flow. Therefore, the static instrumentation tool must implement the appropriate flow functions and strictly follow the rules to model the source code's interaction with the program's data/control flow. SQUIRREL adheres to these rules, and also noteworthy to realize that the SQUIRREL-generated information flow is human-readable, and such a notation is essential for applications with critical information flows.

III. SQUIRREL Source Code Instrumentation

A generic Instrumentation code to parse raw C/CPP files is almost impossible, as the code's syntactic constructs vary extensively. However, SQUIRREL is designed for C source code applications by instrumenting the LLVM IR (the low-level intermediate representation code) generated from the C source code using the Clang compiler. LLVM IR follows a generic format; hence, it can get instrumented without affecting the file syntactically and semantically. Furthermore, it is a typed, single static assessment (SSA) language, representing the compiler optimization and transformation of the program source code. The instrumented IR will be compiled into executables by the Clang compiler. The conversion of source code to LLVM IR, portrays detailed information on each of the variables and functions used in the source code. The disassembled human-readable format of LLVM IR makes it convenient for the instrumentation. Therefore, SQUIRREL instruments the code by finding the DU chain. We identify the DU chain of variables and function calls from the LLVM IR, instrument the SQUIRREL code, extract the memory addresses and other *interesting parameters*[9] to render the precise sequential/control, conditional, and iterative information flow. To acquire this, we should identify various classes of memory regions such as heap, stack, and data and block started by symbol (BSS) segment and also the different data types (primitive/compound) dealt with. A sample of the basic SQUIRREL instrumentation code for C constructs are showed [see Appendix Figure 4]. Likewise, we instrument each critical location in the code by automatically finding it based on DU chain.

To address the various classes of memory regions that facilitate dynamic memory allocations/deallocation (heap) under the header *stdlib.h* are *malloc()*, *realloc()*, *calloc()*, and *free()*. It is more like a free-floating memory region; in LLVM IR, it gets represented with *store* instruction. The data stored in the heap space is accessible or visible to all threads and processes, and thus, it is not as safe as the stack memory space. Furthermore, the heap memory location is not managed automatically by the programmers and is also not tightly managed by the processor. If a programmer does not handle the code for heap memory space, then it is inevitable that a memory leak can happen in the program. Hence, it is significant to know about heap memory space allocation and deallocation and the addresses assigned for the variables used in this space. We require precise knowledge of all variables allocated and deallocated to track the information throughout its scope. We have discussed a sample of SQUIRREL instrumentation code which gets instrumented at the DU chain of variables on the heap in the original code [see Appendix Figure 5]. We extract the *interesting parameters* of heap variables to track their information flow precisely and express it more human-readable for a better understanding in an extensive application.

The stack variable memory representation refers to the local variable storage, which allocates automatic variables in continuous memory space, and whenever a function gets called, it stores the information. In C language, it uses constructs such as *default*, *extern*, *local*, and *global*. The SQUIRREL instrumentation understands that any stack memory representation in C has an associated *alloca* or *load* or *store* instruction with corresponding debug information (line number of a variable in C source code), thereby instrument LLVM IR. The SQUIRREL instrumentation takes adequate steps to bitcast the base address value, display the variable name, and associate the variable type with the size [see Appendix Figure 6]. We also compute the offset address concerning size and base address to retrieve the information of a variable declared with the stack memory representation. The memory regions associated with data or BSS segment (static) can be distinguished as shown [see Appendix Figure 7]. An initialized data segment (*data*) manages the global and static variables initialized in the code. In contrast, an uninitialized data segment (*BSS*) handles all the global and static variables that do not explicitly initialize the program or the variables initialized to zero. Furthermore, in the data and BSS segment, the (static) memory allocated for a variable persists throughout the lifetime of program execution, and it stores variables that are declared global or the variables with static initialization. Static memory representation in C has an associated *alloca* or *load* or *store* instruction with its debug information in the LLVM IR. SQUIRREL instrumentation bitcast the base address values of variables and fetches the variable name and its type with size information. We also compute the offset address and retrieve all information of a variable declared with the static memory representation.

[7] https://blog.quarkslab.com/triton-under-the-hood.html

[8] https://dynamorio.org/

[9] *interesting parameters*: variable name, size, line number, timestamp, scope, and file name/path.

A. Data Types Handling with SQUIRREL

The descriptions/definition column, shows the LLVM IR format for each data type and its bitcasting [see Appendix Figure 4]. The LLVM IR format will use explicit *load* and *store* instructions to access and store the value. We instrument the SQUIRREL code at each assignment and allocation for retrieving the *interesting parameters* out of it. However, in the case of compound data types, we have to handle data structures such as *struct*, *array*, *stack*, *queue*, *linked list*, *hash table*, and *pointers* [14]. We may have sub-divisions for each category. For example, in the case of *pointers*, we have categorizations such as *void*, *wild*, *dangling*, *null*, and *complex pointers* [14]. Therefore, we must understand the initialization, assignment, and allocation instruction in the LLVM IR to handle these. Once SQUIRREL scans through the LLVM IR code, it understands each of those cases and instruments the code required to extract the *interesting parameters*. While running the program, SQUIRREL strictly follows each variable type and extracts the scope and memory addresses of these variables for future purposes. An example is discussed how SQUIRREL instruments the code related to an *array type* [see Appendix Table IV].

B. Memory Address Tracing of Variables with SQUIRREL

SQUIRREL maps the addresses of the variables in the source code. While dealing with tracing addresses of the variables, we have to deal with dynamic, stack/static variables. Handling stack/static variables is similar to the compound data structure handling discussion in section III-A. However, handling dynamically assigned variables and the address mappings of such variables is tricky since we must deal with deallocations during the program control flow at any time.

While handling stack/static variables, we deal with instructions with *alloca*, *load*, and *store*, which is pretty straightforward. SQUIRREL instrumentation codes are written with *variadic* argument lists to handle all data types. However, handling dynamic variable types must address *malloc()*, *realloc()*, and *calloc()*, and *free()*. A hash table is programmed [14] to handle dynamic memory allocations, where SQUIRREL inserts, deletes, and searches for the required variables whenever it is needed while doing the information flow tracking.

IV. SQUIRREL System Modeling

We propose a temporal logic approach to understand the system modeling of SQUIRREL and provide a formal basis for the specification to evaluate it. To explain the system model, we have leveraged the temporal logic such as linear temporal logic (LTL) [31] and computational tree logic (CTL) [32] and quantify and verify the evaluation of the SQUIRREL instrumentation. We try to address the computational path scenarios and specific states in SQUIRREL using the boolean and temporal operators. We evaluate SQUIRREL's safety, liveness, and fairness using LTL and further extend the evaluation with CTL, which takes care of the branching-time logic. We have considered the LLVM IR instruction code such as *allocaInst*, *storeInst*, and *loadInst* along with the instruction code for *malloc()*, *realloc()*, *calloc()*, *free()*, and the function instruction *defineInst*. SQUIRREL's transition system state graph represents $T_{SQUIRREL}$, and that has the vertices and edges as in Equation 1.

$$V = I, E = (i, i^{'}) \in I \times I; i^{'} \in Succ(i)\ ;\ i \in I \tag{1}$$

Where I is the LLVM IR instruction code and $Succ(i)$ is the Successor(i). The state graph of transition system $T_{SQUIRREL}$ has a vertex for each instruction in $T_{SQUIRREL}$ and an edge between the vertices v and $v^{'}$ whenever $i^{'}$ is a direct successor of i in $T_{SQUIRREL}$ for the instrumentation action α. Let *Succ*(i)* denote the states that are reachable in the state graph *G($T_{SQUIRREL}$)* from i. For instruction I, we can represent *Succ*(i)* as in Equation 2.

$$Succ^{*}(I) = \cup_{i \in I} Succ^{*}(i) \tag{2}$$

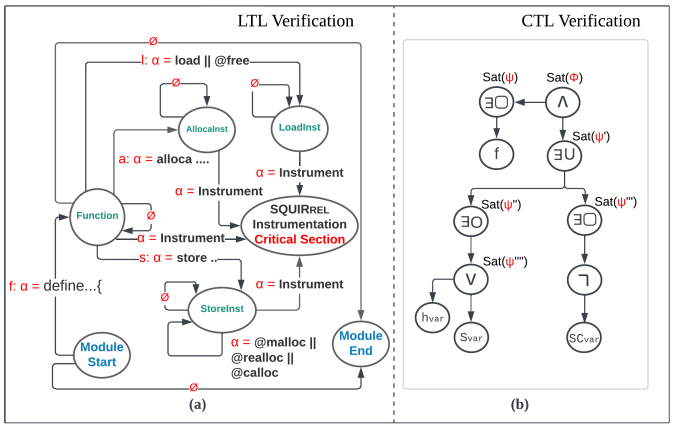


Fig. 1. SQUIRREL LTL and CTL Model Verification.

Having defined the initial constraints, let us verify the LTL properties such as safety, liveness, and fairness.

1. Safety: We introduce the linear temporal properties using a condition ϕ for the states, and it is required that ϕ hold for all the reachable states. $T_{SQUIRREL}$ needs to be safe concerning deadlock, and hence the transition system must adhere to the mutual exclusion property. In general, we represent it using the propositional logic (Equation 3).

$$\phi = \neg crit_1 \vee \neg crit_2\ ;\ crit\ \text{ is critical section of system} \tag{3}$$

The implementation of $T_{SQUIRREL}$ ensures that during instrumentation on *allocaInst*, the instrumentation processes in other section instructions, heap instructions, function definition codes will be in a *wait* state as in Equation 4. It shows that the SQUIRREL instruments over all these instructions invariantly. At once, only one of those instructions enters into the critical section for further instrumentation.

$$\phi = V_i \neg wait_i\ ;\ i\ \text{ is the instruction types} \tag{4}$$

2. Liveness: Following the safety property, $T_{SQUIRREL}$ must also possess the liveness property to reach its goal. Otherwise, the process will not get into the critical section, and it will suffer from starvation. In such a case, information retrieval from the codes of LLVM IR about all variables and functions will never happen. Hence, $T_{SQUIRREL}$ needs to acquire starvation freedom by allowing each waiting process to get into its critical section eventually. During the development of SQUIRREL, we have designed so that each instruction containing *alloca*, *store*, *load*, heap instructions, and function definitions are given entry to critical sections for their corresponding instrumentation sequentially over each block code. We have formulated the liveness property as shown in Equation 5. It reflects that each process in the waiting list will surely jump to the critical section later.

$$\forall j \geq 0 \cdot \wedge_{j}^{i=0} (wait_i \in I_j \Rightarrow (\exists k > j \cdot crit_i \in I_k))\ ;\ i, j, k \in I \tag{5}$$

3. Fairness: Fairness constraints give an action view of the transition system. LTL can verify the synchronization of the selected processes that executes sequentially. It evaluates the SQUIRREL to its effectiveness as a scheduler. $T_{SQUIRREL}$ follows strict fairness, which means every process enabled will get the turn infinitely often. $T_{SQUIRREL}$ is assumed to be finite with no terminal states. The pair of processes in $T_{SQUIRREL(i)}$ and $T_{SQUIRREL(j)}$ should synchronize with its standard sets of actions as in Equation 6, and we assume it as the second part in Equation 6.

$$Syn_{i,j} = Act_i \cap Act_j\ \ \&\ \ Syn_{i,j} \cap Act_k = \emptyset\ ;\ i \neq j\ \&\ \forall k \neq i, j \tag{6}$$

In $T_{SQUIRREL}$, if $\mathcal{P}$ is an LTL property over the instruction set that needs instrumentation and $\mathcal{F}$ a fairness assumption over:

- α = **a (alloca), f (function), s (store), l (load)**, where **afsl** are the instrument options for the binary.
- The instrumentation on a combination of these actions: For example, heap memory information retrieval based *malloc()*, *realloc()*, *calloc()*, and *free()* occurs when the combination of instrument actions on *loadInst* and *storeInst* is made favorable, and hence fairness can be formulated as in Equation 7.

$$T_{SQUIRREL} \models_F P \Leftrightarrow \alpha \mid \alpha \in Syn_{i,j} \; ; 0 < i < j \leq n \tag{7}$$

The illustration of the above properties of LTL is as shown in Figure 1(a). SQUIRREL analyzes the LLVM IR of the benchmark and instruments the necessary SQUIRREL code to it sequentially. If SQUIRREL encounters any of the instructions *i*, (*i* $\in$ *allocInst*, *storeInst*, *loadInst*, *malloc()*, *realloc()*, *calloc()*, *free()*, and *defineInst*) through the action of analyzing α, and then the next phase is instrumentation. We consider the SQUIRREL instrumentation as the critical section in the process. It strictly follows substantial fairness along with safety and liveness property. The $T_{SQUIRREL}$ goes through the instrumentation keeping any one of the instructions *i* in the critical section. It ensures that processes in the *wait* state will eventually fall into the critical section shortly. The SQUIRREL LLVM IR analysis and instrumentation happens synchronously with the initiation of arguments **a**, **f**, **s**, and **l** by the $T_{SQUIRREL}$'s scheduler. The representation of null transition uses ϕ, which denotes there is no active event in the state.

We use the CTL system modeling to verify the SQUIRREL instrumentation concerning the global model checking. Suppose that the CTL formula is given by ϕ, then we check whether the transition in the program induced by after instrumentation is as in Equation 8.

$$T_{SQUIRREL} \models \phi \tag{8}$$

For the verification, we have used the CTL modalities such as $\exists\Box$, $\exists O$, and $\exists U$; where $\Box$ represents *forever*, O represents *next*, and U represents *until*. CTL model checking of SQUIRREL follows the following principles.

- $T_{SQUIRREL}$ is finite and has no terminal states.
- Set *Sat*(ϕ) of all states satisfying ϕ which computes recursively.
- $T_{SQUIRREL} \models \phi \Leftrightarrow I \subseteq Sat(\phi)$; *I* is the initial state wherein the LLVM IR is non-instrumented.

We represent the atomic propositions of SQUIRREL as f: functions, s_{var}: stack variables, h_{var}: heap variables, and sc_{var}: storage class variables. Hence, using the state formula absorbing the existential positive normal form (EPNF), the CTL formula takes the form of Equation 9. It denotes that ϕ is satisfied when the instrumentation happens recursively whenever the function f exists *forever* in the LLVM IR code, eventually *next* with stack variables(s_{var}) or heap variables (h_{var}), until there are no storage class variables (sc_{var}).

$$\phi = \exists\Box f \wedge \exists((Os_{var} \vee Oh_{var})U\exists\Box\neg sc_{var}) \tag{9}$$

Figure 1(b) infers that Equation 8 happens when a satisfied state occurs as in Equation 10 and the sub formulas obtained after decomposition of the state formula in Equation 9.

$$Sat(\phi) = Sat(\psi) \wedge Sat(\psi_{\prime}) \; ; \psi \text{and } \psi_{\prime} \text{are sub formulas} \tag{10}$$

The subsequent phase of work is the query addressing communication, where the server responds with the information for the queried address and counter. This process can have a progress property that every request will eventually lead to a response in the system and formulated as in Equation 11, and shows that always for a request there will be always eventually a response.

$$\Box(request \rightarrow \Diamond response) \; ; \Box \text{ always } \& \; \Diamond \text{ eventually} \tag{11}$$

V. Evaluation

We have selected various benchmark programs that fall into different areas of software applications. The selection of benchmarks was purposeful since we would like to show that SQUIRREL can instrument applications from a comprehensive and diverse set of applications. The selected benchmarks for the evaluation are (1) Nginx (for web serving, reverse proxying, caching, load balancing, and media streaming). (2) OpenSSL (to generate private keys, create CSRs, install SSL/TLS certificate, and identify certificate information). (3) SQLite3 (used as lightweight and access databases using a nonstandard variant of the SQL query language). (4) Libpng (to read, create, and manipulate PNG image files), (5) Coreutils (to work with file, shell, and text manipulation utilities of the GNU OS), (6) Vim (an editor to create and change any text very efficiently), (7) Tar (a standard compression utility), and (8) Snort (a real-time network traffic and packets collection, correlation, and analysis tool). The evaluations are carried out with a 3.40GHz Intel i7 processor and 32GB RAM and run a Ubuntu 20.04 Linux distribution (kernel v5.15 LTS) with compilers gcc v7.5.0, clang v9.0.1 and LLVM v9.0.1 and no particular application running in the background.

A. Code Coverage Statistics

SQUIRREL instruments after each *alloca/store/load* instruction to obtain the variables' names; hence, it will have a constant time complexity of $\mathcal{O}(1)$. Therefore, for k variables, the complexity would be $k * \mathcal{O}(1)$. Even for dynamic allocation structures, SQUIRREL does not use branching instructions; hence, the overall complexity will be $\mathcal{O}(1)$. However, as we keep the information about dynamically allocated and freed variables' names in the hash table, the time complexity for this operation is $\mathcal{O}(\log n)$ and the space complexity of $\mathcal{O}(n)$, where n is the number of elements.

We assume that the code coverage and after-instrumentation overhead of SQUIRREL instrumentation is a minor concern since we perform code injection statically. However, to analyze each benchmark's code coverage and after-instrumentation overhead, we have measured the time to instrument each benchmark listed in Table I. To understand this, we have to quantify the number of C files and lines of C code that SQUIRREL checks and the lines of LLVM IR code SQUIRREL instruments. The time taken to instrument provides an insight into the instrumentation speed of SQUIRREL in each benchmark. Table I details the time employed for code coverage and to the finish instrumentation on each benchmark and shows that it is reasonable. Dynamic taint tracking or analysis tools do not worry about the binary size increase. However, the instrumented binary size or the instrumentation runtime overhead can be demanding for embedded devices due to the tight hardware resource utilization cost. Therefore, binaries with less runtime overhead are expected since the registers and the counters will rapidly surpass the limits of the memory allocated. Nevertheless, a device that supports LLVM-based program analysis tools must have a reasonable memory availability; hence we expect that the increase in the size of an instrumented binary will not affect it. Yet, we have computed the increase in binary size to examine the SQUIRREL instrumented binary heaviness. As shown in Table I, the binary file size of benchmarks has increased between 21.5% to 69.2% compared with the original binaries in an average of 42.1%. As discussed above, the increment in the binary size will not affect the performance of a device with adequate memory availability. Thus we subject the instrumentation overhead to the increase in the execution time or the binary size of the instrumented applications. It happens because of the injected code, which takes some of the runtime analysis features for the smooth execution of the application. However, we know that one can easily add trivial (instrumentation) code to LLVM IR with a bare minimum overhead, but it could

TABLE I
CODE COVERAGE AND AFTER-INSTRUMENTATION STATISTICS

Benchmarks	TCF	Lines of Code (LOC)		BGI	LOC AI	TI Per LL Line (mS)	TTI (S)	Size of Binary (KB)		Increase in Binary Size (%)
		C Code	LLVM IR Code					Org_Bin	SQI_Bin	
Nginx	198	120764	1051010	2	2158020	0.97	1024	3500	4500	28.6
OpenSSL	928	263215	10196768	182	22397737	0.75	99438	1300	2200	69.2
SQLite3	338	720017	3496710	12	7177830	7.73	27030	3100	4500	45.2
Libpng	71	58726	88430	9	188204	0.45	042	50.2	77.1	53.6
Coreutils	2878	345294	1988926	116	4161975	0.38	764	77.2	109.4	41.7
Vim	182	348533	2156181	3	4512578	6.59	14218	9300	11300	21.5
Tar	234	53043	139327	2	281683	2.03	283	850.7	1300	52.8
Snort	339	202695	632742	3	1295101	7.42	4695	5600	7600	35.7
G.Mean	**341.09**	**190563.9**	**1028576.9**	**9.75**	**2144538.1**	**1.77**	**2528.1**	**1116.9**	**1595.3**	**41.07**

TCF (Total No. of C Files), BGI (No. of Binaries Getting Instrumented [Automatic Selection of Files Needed for Instrumentation from MAKEFILE]), AI (LOC After Instrumentation on LLVM IR), TI (Time to Instrument Per LLVM IR Line [milli Seconds]), TTI (Total Time to Instrument [Seconds]), Org_Bin/SQI_Bin (Original Binary/SQUIRREL Instrumented Binary), G.Mean (Geometric Mean)

impact the execution of an application. Therefore, we want to give importance to spin around the following research question.

- Impact of SQUIRREL instrumentation concerning memory profiling, runtime profiling, and runtime memory address tracing.

B. RQ1. Memory Profiling Overhead

Table II summarizes the upsurge in the heap and stack memory profiling on each benchmark. We have supplied different payloads for the benchmarks to evaluate the deviations in memory profiling while running the original and instrumented binaries. For collecting and comparing the heap and stack memory profiling, heaptrack-/stackusage, Valgrind-massif_heap/stack, and gperftools_heap/stack has been deployed at process execution termination. Table II shows that, except for some benchmarks (OpenSSL, SQLite3, Libpng, and Snort), the heap memory usage deviations are less. The benchmarks with memory profiling upsurge are highly *malloc()* intensive, and the increase (peak) in memory usage comes mainly from the *malloc()* basic blocks. The reason for such an upsurge is due it the instrumentation heaviness on the heap memory calls code blocks. Furthermore, we could observe that in some cases, the SQUIRREL instrumentation code struggled to locate where to release the heap-allocated memory, inturns caused exhaustion in accessing the memory. Therefore, failing to instrument *free()* at some of the instrumented locations turned into an extraneous memory leak. Nevertheless, concerned with the upsurge in stack memory usage, none of the profiling tools reported any consequential irregularities. Also, in some cases (Vim and Snort), there were problems due to the ineptitude in writing to a *.prof* file and segmentation fault since accessing a fault address. Overall, the instrumented binary memory profiling exhibits an average increase of 197% (peak heap memory use).

C. RQ2: Runtime Profiling Overhead

We have profiled the runtime (execution) overhead of SQUIRREL and compared with the original binary, Valgrind instrumented binary, and Nulgrind (the minimal valgrind instrumentation tool). In Table II, we have summarized the various loads supplied for each benchmark. To analyze the overhead, we have averaged the runtime overhead of 10 executions with each load for the corresponding binary.

With the diverse payload provided, we expected an extra runtime overhead in the SQUIRREL instrumented binary since it is inevitable due to the instrumentation. Figure 2 shows the analysis of runtime overhead on instrumented binaries. The runtime overhead of SQUIRREL instrumented binary (Figure 2) shows an upsurge of 197.5% and 188.2% in the case of Snort and SQLite3. Such an upsurge is due to the heaviness in the C code structure of Snort. Snort code uses multi-threading where each worker thread gets operated in an infinite event loop. This code structure complexity can even reflect the time to complete SQUIRREL code instrumentation on Snort binary. Table I shows that it is taking 7.42mS to instrument per LLVM IR line. The multithreaded code loop structure helps fetch and process sequential incoming packets while running the snort. When the SQUIRREL code gets instrumented in this loop structure code, the complexity increases since the heaviness of processing instrumented code gets higher while running multiple times. Also, in the Snort code, we can see an abundance of linking of the dynamic library (seven library dependencies), and the libDAQ library, which is used for network packet acquisition, is statically linked. Some libraries are also written as shared-library or Lua-script plugins, which get loaded while Snort startup. Since the SQUIRREL instrumentation contains some loading code calls, calls to these libraries get delayed, which imposes the runtime overhead. In the case of SQLite3, the increase in runtime is due to the prolonged running loops for query and calls many sub-functions in those loops in SQLite3. Therefore, the aesthetics of the C code structure can result in a very complex LLVM IR code that, in turn, reflects in the SQUIRREL instrumentation and can expect an upsurge in the execution runtime overhead of an average of 40-45%.

Nevertheless, it is way out of the purview when we compare the runtime overhead of the SQUIRREL instrumented with the Valgrind instrumented binary. We can see an average upsurge of 170% while using Valgrind instrumentation and 112% with Nulgrind. Nulgrind is a minimal Valgrind instrumentation option used for debugging source code. There is no instrumentation in that case, but as per the Valgrind documentation[10], it will run roughly 5x more slowly than the original binary. In our benchmark experiment base, it was running an average of 50x slow compared to the original binary execution and an average of 8x slow compared to the SQUIRREL instrumented binary. However, there is an exception in this case concerning the Snort binary due to the reasons explained earlier. There is an increase of 46% runtime overhead on the SQUIRREL instrumented Snort compared with the Valgrind instrumented Snort binary. As we discussed earlier, the SQUIRREL instrumentation occurs inside the loop structure codes of Snort. The complexity increases while processing instrumented code multiple times and is a drawback since we do not call an event STOP signal in SQUIRREL instrumentation. Overall, SQUIRREL raises only a reasonable runtime overhead compared to the runtime of the original binary. In contrast, SQUIRREL performed far better than the state-of-the-art Valgirnd instrumented binary runtime overhead.

D. RQ3. Runtime Memory Address Tracing

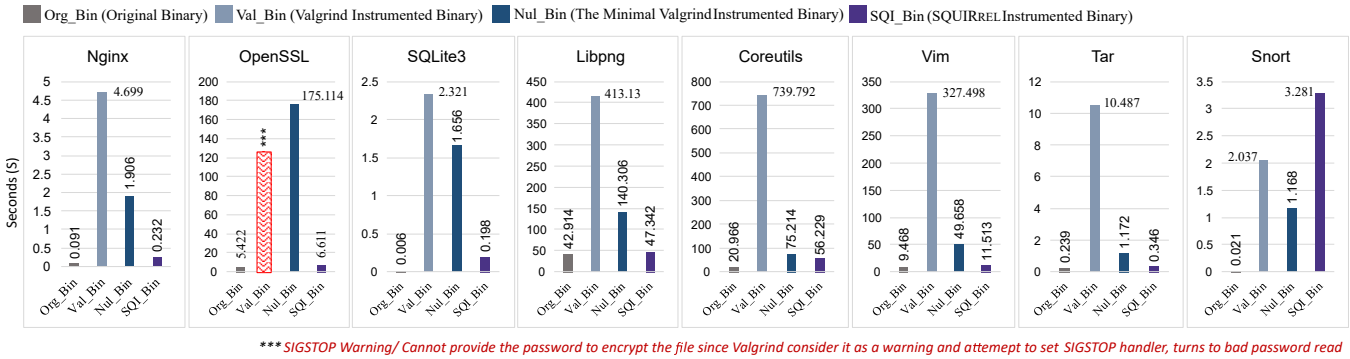
Let us take a simple code snippet (Listing 1) with a buffer overflow to demonstrate the SQUIRREL memory address resolver and runtime memory address tracing.

[10] https://valgrind.org/docs/manual/nl-manual.html

TABLE II
MEMORY PROFILING OF ORIGINAL BINARY AND SQUIRREL INSTRUMENTED BINARY

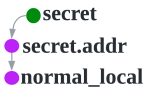
Memory Profiling	*Heaptrack _heap*		*Stackusage _stack*		*Valgrind-massif _heap*		*Valgrind-massif _stack*		*gperftools _heap*		*gperftools _stack*	
Binary	**Org_Bin**	**SQI_Bin**	**Org_Bin**	**SQI_Bin**	**Org_Bin**	**SQI_Bin**	**Org_Bin**	**SQI_Bin**	**Org_Bin**	**SQI_Bin**	**Org_Bin**	**SQI_Bin**
Nginx	3.12M	3.13M	6.23K	7.93K	2.84M	2.96M	268.3K	273.5K	3.1M	3.19M	2.16K	4.32K
OpenSSL	198.24K	67.34M	22.62K	24.34K	168.4K	63.43M	18.66K	23.40K	0.2M	63.4M	0.1M	24.58K
SQLite3	326.35K	1.36G	263.21K	275.85K	257.8K	1.176G	273.1K	288.2K	0.3M	614.8M	0.3M	541.8K
Libpng	445.46K	10.15G	4.29K	12.78K	364.5K	10.82G	2.104K	11.85K	716.67K	11.41G	12.29K	57.34K
Coreutils	113.89K	114.95K	3.29K	5.13K	212.2K	213.9K	5.01K	6.73K	0.1M	0.1M	0.0M	0.0M
Vim	477.73K	502.89K	21.52K	23.96K	416.7K	476.8K	514.8K	611.8K	*Warning***			
Tar	115.24K	115.72K	14.24K	14.94K	43.53K	44.12K	42.41K	48.24K	0.1M	0.1M	14.34K	14.36K
Snort	13.59M	12.25G	13.46K	28.34K	12.90M	*SIGSEGV**	70.55K	*SIGSEGV**	14.2M	11.2G	10.95K	14.25K

*Valgrind crash [accessed a faulting address (0x0) to read/write], **Vim: [Warning: Output is not to a terminal since we want to write it to a .prof file, but cannot generate the .prof file]; **Payloads**: **Nginx**-*process requests*, **OpenSSL**-*encrypt & decrypt with AES-256-CBC (1GB file)*, **SQLite3**-*create DB, table, & perform various operations*, **Libpng**-*test (pngcheck) a PNG image (54MB file)*, **Coreutils**-*utilities commands*, **Vim**-*open a file of 100MB size*, **Tar**- *Ubuntu iso image Compress & decompress*, **Snort**-*start with data acquisition modules (DAQ)*; Org_Bin/ SQI_Bin (Original Binary/ SQUIRREL Instrumented Binary)

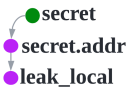


*** SIGSTOP Warning/ Cannot provide the password to encrypt the file since Valgrind consider it as a warning and attempt to set SIGSTOP handler, turns to bad password read

Fig. 2. Performance Overhead Comparison



(a) Track taint source 'secret': Propagation path in normal execution



(b) Track taint source 'secret': Propagation path while buffer overflow occured

Fig. 3. Propagation Paths and Runtime Address Mapping

TABLE III
RESOLVED MEMORY ADDRESSES

Benchmarks	Nginx	OpenSSL	SQLite3	Libpng
RA/TA	659/670	1315/1324	990/994	543/551
Benchmarks	Coreutils	Vim	Tar	Snort
RA/TA	789/792	981/992	592/597	512/529
RA/TA (SQUIRREL Resolved/Total Memory Addresses Allocated)				

```
1  #define BUF_SIZE 10
2  void normal (int secret){ int normal_local = secret;}
3  void leak (int secret){ int leak_local = secret;}
4  int main (int argc, char **argv){
5    int secret, i, iter, ptr_val, buf[BUF_SIZE], canary;
6    void (*func)(int);
7    iter = atoi(argv[2]); ptr_val = atoi(argv[2]);
8    func = &normal; canary = 123;
9    for (i=0; i<iter; i++){ buf[i] = ptr_val;}
10   func(secret); return 0;
11 }
```

Listing 1. Code Snippet for Dynamic Variable Memory Address Mapping

Suppose we set *secret* as a taint source, and we want to observe the taint data flow while running the program. Here the values of *iter* and *ptr_val* depend on the user's input. If *iter* does not exceed the size of the *buf*, then the program will run without any fault, as shown in Figure 3a; otherwise, if the size of *iter* is larger than the size of *buf* allotted, then line number 9 will overwrite the adjacent variables *canary* and *func* with *ptr_val*. Furthermore, if *ptr_val* is assigned as the function address of *leak()*, then the code will generate a function leak in line number 10, as shown in Figure 3b. Therefore, in this code snippet, the number of memory addresses resolved with SQUIRREL instrumentation in the "normal"case is 8/9 since *leak_local* is not getting called; otherwise, without *normal_local*. In addition, we have computed the ratio of variables whose memory addresses are successfully resolved in all the benchmarks listed in this work. Table III shows the number of resolved addresses in each of the benchmarks. It resonates that SQUIRREL instrumentation can resolve a reasonable number of variables' memory addresses in each benchmark with a percentile of 97-99%.

VI. CONCLUSION

We have presented a configurable static code instrumentation tool SQUIRREL, a variable-level online *dynamic information flow tracking* (DIFT) with enriched semantics and source-level logging of memory allocation events to enhance the interpretation. We have analyzed the performance by instrumenting various real-world programs, which shows an average upsurge of only 40-45%. However, the memory leaks and peak memory use with instrumented binaries, showing an average increase of 197%, and confirmed it with various memory analysis tools. We have realized the cases of memory leaks, and they will get fixed in the future version. The runtime memory address tracing of the instrumented binaries shows 97-99% efficiency. Overall, SQUIRREL information flow tracking improves the readability of the end users/ system administrators, which is essential for applications concerning security forensics and malware analysis.

Acknowledgment

This study is supported by the National Science and Technology Council of the Republic of China under grant numbers 111-2628-E-A49-007-MY2 and 111-2218-E-A49-013-MBK

References

[1] P. Bhattacharya and I. Neamtiu, "Assessing programming language impact on development and maintenance: A study on c and c++," in *Proc. of the 33rd International Conf. on Software Engg. (ICSE)*. ACM, 2011, pp. 171–180.

[2] D. Spinellis, P. Louridas, and M. Kechagia, "The evolution of C programming practices: A study of the Unix operating system 1973–2015," in *Proc. of the 38th International Conf. on Software Engg. (ICSE)*. ACM, 2016, pp. 748–759.

[3] Y. Roumani, "Patching zero-day vulnerabilities: an empirical analysis," *Journal of Cybersecurity*, vol. 7, no. 1, Nov 2021.

[4] J. Lambert, "Microsoft digital defense report 2022," Microsoft Research, Redmond, WA, USA, Tech. Rep. Defense Report 2022, 2022. [Online]. Available: https://www.microsoft.com/en-us/security/business/microsoft-digital-defense-report-2022-nation-state-attacks

[5] G. J. Holzmann, "The power of 10: Rules for developing safety-critical code," *Computer*, vol. 39, no. 6, pp. 95–97, Jun 2006.

[6] B. Zeng, G. Tan, and G. Morrisett, "Combining control-flow integrity and static analysis for efficient and validated data sandboxing," in *Proc. of the 18th ACM Conf. on Computer and Communications Security (CCS)*. ACM, 2011, pp. 29–40.

[7] P. Muntean, M. Neumayer, Z. Lin, G. Tan, J. Grossklags, and C. Eckert, "Analyzing control flow integrity with LLVM-CFI," in *Proc. of 35th Annual Computer Security Applications (ACSAC)*. ACM, 2019, pp. 584–597.

[8] M. Abadi, M. Budiu, U. Erlingsson, and J. Ligatti, "Control-flow integrity principles, implementations, and applications," *ACM Trans. Inf. Syst. Secur.*, vol. 13, no. 1, Nov 2009.

[9] C. Zhang, T. Wei, Z. Chen, L. Duan, L. Szekeres, S. McCamant, D. Song, and W. Zou, "Practical control flow integrity and randomization for binary executables," in *IEEE Symposium on Security and Privacy (S&P)*, 2013, pp. 559–573.

[10] S. Lee, H. Kang, J. Jang, and B. B. Kang, "SaVioR: Thwarting stack-based memory safety violations by randomizing stack layout," *IEEE Trans. on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2559–2575, Mar 2022.

[11] B. Gui, W. Song, H. Xiong, and J. Huang, "Automated use-after-free detection and exploit mitigation: How far have we gone?" *IEEE Trans. on Software Engg.*, vol. 48, no. 11, pp. 4569–4589, Nov 2022.

[12] E. Iannone, R. Guadagni, F. Ferrucci, A. De Lucia, and F. Palomba, "The secret life of software vulnerabilities: A large-scale empirical study," *IEEE Trans. on Software Engg.*, vol. 49, no. 1, pp. 44–63, Jan 2023.

[13] Z. Xu, "Source code and binary level vulnerability detection and hot patching," in *Proc. of the 35th IEEE/ACM International Conf. on Automated Software Engg. (ASE)*. ACM, 2021, pp. 1397–1399.

[14] S. Mallissery, K. Chiang, C. Bau, and Y. Wu, "Pervasive micro information flow tracking," *IEEE Trans. on Dependable and Secure Computing*, vol. Early Access, pp. 1–18, Jan 2023.

[15] M. C. Neto, R. P. A. Araujo, M. L. Chaim, and J. Offutt, "Graph representation for data flow coverage," in *IEEE 45th Annual Computers, Software, and Applications Conf. (COMPSAC)*, 2021, pp. 952–961.

[16] A. R. Bernat and B. P. Miller, "Anywhere, any-time binary instrumentation," in *Proc. of ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools (PASTE)*. ACM, 2011, pp. 9–16.

[17] P. D. Schubert, P. Gazzillo, Z. Patterson, J. Braha, F. Schiebel, B. Hermann, S. Wei, and E. Bodden, "Static data-flow analysis for software product lines in C: Revoking the preprocessor's special role," *Automated Software Engg.*, vol. 29, no. 1, May 2022.

[18] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, "Pin: Building customized program analysis tools with dynamic instrumentation," *SIGPLAN Not.*, vol. 40, no. 6, pp. 190–200, Jun 2005.

[19] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," *SIGPLAN Not.*, vol. 42, no. 6, pp. 89–100, Jun 2007.

[20] D. C. D'Elia, E. Coppa, S. Nicchi, F. Palmaro, and L. Cavallaro, "SoK: Using dynamic binary instrumentation for security (and how you may get caught red handed)," in *Proc. of the ACM Asia Conf. on Computer and Communications Security (Asia CCS)*. ACM, 2019, pp. 15–27.

[21] V. P. Kemerlis, "libdft: Dynamic data flow tracking for the masses," in *Proc. of the 38th Annual Computer Security Applications Conf. (ACSAC)*. ACM, 2022, pp. 1–5.

[22] R. J. Rodríguez, J. A. Artal, and J. Merseguer, "Performance evaluation of dynamic binary instrumentation frameworks," *IEEE Latin America Trans.*, vol. 12, no. 8, pp. 1572–1580, Dec 2014.

[23] D. C. D'Elia, L. Invidia, F. Palmaro, and L. Querzoni, "Evaluating dynamic binary instrumentation systems for conspicuous features and artifacts," *Digital Threats*, vol. 3, no. 2, Feb 2022.

[24] Z. Chen, Q. Zhang, J. Wu, J. Yan, and J. Xue, "A source-level instrumentation framework for the dynamic analysis of memory safety," *IEEE Trans. on Software Engg.*, pp. 1–21, Sep 2022.

[25] P. D. Schubert, R. Leer, B. Hermann, and E. Bodden, "Know your analysis: How instrumentation aids understanding static analysis," in *Proc. of the 8th ACM SIGPLAN International Workshop on State Of the Art in Program Analysis (SOAP)*. ACM, 2019, pp. 8–13.

[26] S. Chen, Z. Lin, and Y. Zhang, "SelectiveTaint: Efficient data flow tracking with static binary rewriting," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Assoc., Aug. 2021, pp. 1665–1682.

[27] Y. Sui and J. Xue, "SVF: Interprocedural static value-flow analysis in LLVM," in *Proc. of the 25th International Conf. on Compiler Construction (CC)*. ACM, 2016, pp. 265–266.

[28] Q. Shi, X. Xiao, R. Wu, J. Zhou, G. Fan, and C. Zhang, "Pinpoint: Fast and precise sparse value flow analysis for million lines of code," *SIGPLAN Not.*, vol. 53, no. 4, pp. 693–706, Jun 2018.

[29] M. Harman and P. O'Hearn, "From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis," in *2018 IEEE 18th International Working Conf. on Source Code Analysis and Manipulation (SCAM)*, Sep 2018.

[30] A. Eghbali and M. Pradel, "DynaPyt: A dynamic analysis framework for Python," in *Proc. of 30th ACM Joint European Software Engg. Conf. & Symposium on the Foundations of Software Engg. (FSE)*. ACM, 2022.

[31] Y. Gao, A. Abate, F. J. Jiang, M. Giacobbe, L. Xie, and K. H. Johansson, "Temporal logic trees for model checking and control synthesis of uncertain discrete-time systems," *IEEE Trans. on Automatic Control*, vol. 67, no. 10, pp. 5071–5086, Oct 2022.

[32] U. Boker and Y. Shaulian, "Automaton-based criteria for membership in CTL," in *Proc. of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. ACM, 2018, pp. 155–164.

Appendix

A. LLVM IR and SQUIRREL Sample Instruction Sets

TABLE IV
LLVM IR Instructions of an Array

C Code	LLVM IR Instruction
int i=0 int empID[5]	%i = alloca i32, align 4 %empID = alloca [5 x i32], align 16
empID[0]=10280	%arrayidx = getelementptr inbounds [5 x i32], [5 x i32]* %empID, i64 0, i64 0, !dbg !18 store i32 10280, i32* %arrayidx, align 16, !dbg !19
empID[1]=10260	%arrayidx1 = getelementptr inbounds [5 x i32], [5 x i32]* %empID, i64 0, i64 1, !dbg !20 store i32 10260, i32* %arrayidx1, align 4, !dbg !21

From Table IV, we can infer that the initialization of an *array* takes the form of *alloca* instruction. The *array* elements occupy the memory spaces sequentially with the help of *store* instruction. The instruction *%arrayidx* and *%arrayidx1* are the local register variables names from LLVM IR that are implications of *array[0]* and *array[1]*. SQUIRREL thus performs the instrumentation procedure required for the *alloca* and the *store* instruction. To apprehend the *arrayidx* names in the *store* instruction, SQUIRREL traces back to the previous *getelementptr inbounds* instruction to get the actual variable name, which is the second argument in the latter. From Table IV, *%arrayidx1 = getelementptr inbounds [5 x i32], [5 x i32]* %empID, i64 0, i64 1, !dbg !20* clearly reflects that *%arrayidx1* is nothing other than its second argument *%empid*. Similarly, we handle each other data structures in the compound data types by understanding the LLVM IR instructions associated with and the SQUIRREL instrument necessary code for each based on the category.

A Sample of SQUIRREL Instrumentation Instruction Set

Component to Retrieve	Corresponding C Code	LLVM IR SQUIRREL Instrumentation Instructions		
		Initialization Segment of LLVM	After the Corresponding Instruction of Variable/Function	Description/ Definition
Name of Variable	(a);	@.str = private unnamed_addr constant [2 x i8] c"a\00", align 1	%call = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([2 x i8], [2 x i8]* @.str, i64 0, i64 0))	The strings to *stdout* get stored in the global variables after the module declaration section in LLVM IR. We mark those global variables appended with *unnamed_addr*, which indicates only the content is significant and not the address. Similarly, variable names, function names, file names, and line numbers append in the same way as showed in next rows.
Address of Variable	("%p", &a);	@.str1 = private unnamed_addr constant [3 x i8] c"%p\00", align 1	%call1 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([3 x i8], [3 x i8]* @.str1, i64 0, i64 0), i32* %a)	
Size of Variable	("%ld", (void*) (&a+1)-(void *)(&a));	@.str.2 = private unnamed_addr constant [4 x i8] c"%ld\00", align 1	%add.ptr = getelementptr inbounds i32, i32* %a, i64 1 %0 = bitcast i32* %add.ptr to i8* %1 = bitcast i32* %a to i8* %sub.ptr.lhs.cast = ptrtoint i8* %0 to i64 %sub.ptr.rhs.cast = ptrtoint i8* %1 to i64 %sub.ptr.sub = sub i64 %sub.ptr.lhs.cast %sub.ptr.rhs.cast %call2 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([4 x i8], [4 x i8]* @.str.2, i64 0, i64 0), i64 %sub.ptr.sub) ret i32 0	The *alloca* instruction allocates *sizeof(<type>)*NumElements* bytes of memory on the runtime stack, returning a pointer of the appropriate type to the program. If *NumElements* is specified, it mentions the number of elements allocated. The *store* instruction to write into a memory location. It takes two arguments,The first argument is the value (i.e., to write), and the second argument contains the address of the variable, where the value resides. We log the second argument to aid the heap data structure where the *store* instruction details the value stored in each memory location.
Line Number	("%d", __LINE__);	@.str.3 = private unnamed_addr constant [3 x i8] c"%d\00", align 1	%call3 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([3 x i8], [3 x i8]* @.str.3, i64 0, i64 0), i32 9)	
Function Name	("%s()",__ FUNCTION__);	@.str.4 = private unnamed_addr constant [5 x i8] c"%s()\00", align 1 @__FUNCTION__.main = private unnamed_addr constant [5 x i8] c"main\00", align 1	%call4 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([5 x i8], [5 x i8]* @.str.4, i64 0, i64 0), i8* getelementptr inbounds ([5 x i8], [5 x i8]* @__FUNCTION__.main, i64 0, i64 0))	LLVM considers global variables as pointers. Hence for an explicit dereference of the global variable, the *load* instruction is required. The second argument in every *load* instruction contains the address of the variable. The *load* instruction helps load the content of the type from the address used by the operator into the variable. In contrast, the *getelementptr* instruction involves structure field access, array indexing, array-based access, and pointer arithmetic.
File Name	("%s()",__FILE __);	@.str.5 = private unnamed_addr constant [3 x i8] c"%s\00", align 1 @.str.6 = private unnamed_addr constant [7 x i8] c"test.c\00", align 1	%call5 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([3 x i8], [3 x i8]* @.str.5, i64 0, i64 0), i8* getelementptr inbounds ([7 x i8], [7 x i8]* @.str.6, i64 0, i64 0)) ret i32 0	
Timestamp	("%d", (int) time(NULL));	@.str.3 = private unnamed_addr constant [3 x i8] c"%d\00", align 1	%call6 = call i64 @time(i64* null) #3 %conv = trunc i64 %call6 to i32 %call7 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([3 x i8], [3 x i8]* @.str.3, i64 0, i64 0), i32 %conv)	Types (Representations) i8 ; (8/8 = 1 byte) " char , u_char " i16 ; (16/8 = 2 bytes) " short , u_short " i32 ; (32/8 = 4 bytes) " int , u_int " i64 ; (64/8 = 8 bytes) " long , u_long " i32 ; (32/8 = 4 bytes) " float " i64 ; (64/8 = 8 bytes) " double " x86_fp80 ; (128/8 = 16 bytes) " long double " [4 x i32] ; (32*4/8 = 16 bytes) " int array "
Address of Function	("%p", main);	@.str1 = private unnamed_addr constant [3 x i8] c"%p\00", align 1	%call8 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([3 x i8], [3 x i8]* @.str.1, i64 0, i64 0), i32 (...)* bitcast (i32 ()* @main to i32 (...)*))	

Fig. 4. Sample Set of SQUIRREL Instrumentation Instructions.

A Sample of SQUIRREL Instrumentation Instruction Set for Various Classes of Memory Regions (Heap)

Classes of Memory Regions	Representation	LLVM IR SQUIRREL Instrumentation Instructions		
		C Code Snippet	SQUIRREL Instrumented	Description/ Definition
malloc()	ptr = (castType*) malloc(size); <result> = malloc <type>, uint <NumElements>;	int *p = (int*)malloc(sizeof(int)*4);	%call = call noalias i8* @malloc(i64 16) #3, !dbg !31 %0 = bitcast i8* %call to i32*, !dbg !32 store i32* %0, i32** %p, align 8, !dbg !30	The instruction *@malloc(i64 16)* provides the information that the *typesize* is 16, and the instruction *%call* refers to the initial address for the allocation. The *store* instruction following the *malloc* has the dynamic variable name.
realloc()	realloc (void* ptr, size_t size); realloc(base_address, typesize);	ptr = realloc(ptr, n * sizeof(int));	%mul = mul i64 %conv19, 4, !dbg !78 %call20 = call i8* @realloc(i8* %18, i64 %mul) #5, !dbg !79 %20 = bitcast i8* %call20 to i32*, !dbg !79 store i32* %20, i32** %ptr, align 8, !dbg !80	The initialization of variable *ptr* with a base address value of *%18*, and *%mul* is its *typesize*.
calloc()	ptr = (cast-type*)calloc(n, element-size); calloc (size_t __nmemb, size_t __size);	ptr = (int*)calloc(n, sizeof(int));	%18 = load i32, i32* %n, align 4, !dbg !76 %conv18 = sext i32 %18 to i64, !dbg !76 %call19 = call noalias i8* @calloc(i64 %conv18, i64 4) #5, !dbg !77 %19 = bitcast i8* %call19 to i32*, !dbg !78 store i32* %19, i32** %ptr, align 8, !dbg !79.	The instruction *%call19* is the base address, and *i64 4* is the size. It consists of *n* elements with *typesize* 4.

free()	free(ptr);
	free(base_address);
		int *ptr = (int*)malloc(sizeof(int)*4);
		if (foo != NULL) --------
		free(ptr);
			%11 = load i32*, i32** %ptr, align 8, !dbg !44
			%12 = bitcast i32* %11 to i8*, !dbg !44
			call void @free(i8* %12) #3, !dbg !45
				Here, *%12* is the initial address get freed. The *load* instruction preceding the *free()* call contains the name of the dynamic variable *ptr*.

Fig. 5. Sample Set of SQUIRREL Instrumentation Instructions for Classes of Memory Regions (Heap)

A Sample of SQUIRREL Instrumentation Instruction Set for Various Classes of Memory Regions (Stack)

Classes of Memory Regions	C Code Snippet	LLVM IR SQUIRREL Instrumentation Instructions		
		LLVM IR	SQUIRREL Instrumented	Description/ Definition

Default	auto int val{
	auto int val;
	....
	}
		%val = alloca i32, align 4
		store i32 10, i32* %val, align 4
		....
			@fstring1 = private unnamed_addr constant [5 x i8] c"val\00", align 1
			%cast1 = bitcast i32* %val to i8*
			call void @variable_capture(i8* %cast1, i8* getelementptr inbounds ([5 x i8], [5 x i8]* @fstring1, i32 0, i32 0), i64 %sizeofvar1)
				Varibale name from *alloca* is assigned to *@fstring1* and defined globally. Bitcasted the address value from *i32* to *i8** and asssigned to *%cast1*. Computed the size from *load* instruction. Here it is *i32*, hence size is 4 and assigned it to *sizeofvar1*.

Extern	extern int ext;	@ext = dso_local global i32 48, align 4 .... %2 = load i32, i32* @ext, align 4	@lstring1 = private unnamed_addr constant [5 x i8] c"ext\00", align 1 %lcast1 = bitcast i32* @ext to i8* call void @variable_capture(i8* %lcast1, i8* getelementptr inbounds ([5 x i8], [5 x i8]* @lstring1, i32 0, i32 0), i64 %lsizeofvar1)	Varibale name is assigned to *@lstring1* and defined globally. Bitcasted the address value from *i32* to *i8** and asssigned to *%lcast1*. Captured variable name from the second argument of *load* and assigned to *lstring1*. Computed the size from *load* instruction. Here it is *i32,* hence size is 4 and assigned to *lsizeofvar1*.

Local	void foo (void){
	static int ret;
	// increment ret
	...
	}
		define dso_local void @foo() { entry:
		@foo.ret = internal global i32 13, align 4
		%inc = add nsw i32 %0, 1
		%0 = load i32, i32* @foo.ret, align 4
			@lstring2 = private unnamed_addr constant [9 x i8] c"foo.ret\00", align 1
			%lcast2 = bitcast i32* @foo.ret to i8*
			!%lsizeins2 = getelementptr inbounds i32, i32* @foo.ret, i64 1
			call void @variable_capture(i8* %lcast2, i8* getelementptr inbounds ([9 x i8], [9 x i8]* @lstring2, i32 0, i32 0), i64 %lsizeofvar2)
				Varibale name is assigned to *@lstring2* and defined globally. Bitcasted the address value from *i32* to *i8** and asssigned to *%lcast2*. Captured variable name from the second argument of *load* and assigned to *lstring2*. Computed the size from *load* instruction. Here it is *i32*, hence size is 4 and assigned to *lsizeofvar2*.

Global	static int cnt;
	int main (){
	....
	}
		@cnt = internal global i32 5, align 4
		define dso_local i32 @main() { entry:
		%3 = load i32, i32* @cnt, align 4
		%inc = add nsw i32 %3, 1
			@lstring3 = private unnamed_addr constant [5 x i8] c"cnt\00", align 1
			%lcast3 = bitcast i32* @cnt to i8*
			!%lsizeins3 = getelementptr inbounds i32, i32* @cnt, i64 1
			call void @variable_capture(i8* %lcast3, i8* getelementptr inbounds ([5 x i8], [5 x i8]* @lstring3, i32 0, i32 0), i64 %lsizeofvar3)
				Varibale name is assigned to *@lstring3* and defined globally. Bitcasted the address value from *i32* to *i8** and asssigned to *%lcast3*. Captured variable name from the second argument of *load* and assigned to *lstring3*. Computed the size from *load* instruction. Here it is *i32*, hence size is 4 and assigned to l*sizeofvar3*.

Fig. 6. Sample Set of SQUIRREL Instrumentation Instructions for Classes of Memory Regions (Stack)

A Sample of SQUIRREL Instrumentation Instruction Set for Various Classes of Memory Regions (Static)

Classes of Memory Regions	C Code Snippet	LLVM IR SQUIRREL Instrumentation Instructions		
		LLVM IR	SQUIRREL Instrumented	Description/ Definition

Initialized static variable stored in data segment	int main (){
	static int val = 4;
	foo (val);
	return 0;
	}
		%val.addr = alloca i32, align 4
		%saved_stack = alloca i8*, align 8
		store i32 %val, i32* %val.addr, align 4
			%storecast1 = bitcast i32* %val.addr to i8*
			call void @variable_capture(i8* %storecast1, i8* getelementptr inbounds ([10 x i8], [10 x i8]* @storeVar1, i32 0, i32 0), i64 %sizeofstoreVar1, i32 0)
				Bitcasted the address from *i32* to *i8** and assign to storecast1. Captured Variable name *val* from the first parameter of *store* and define *@storeVar1* globally. Compute the size from *load* instruction. Here it is i32, hence size is 4 and assign to *%loadsizeofVar1*.

Uninitialized static variable stored in BSS	void foo (int val){
	static int seed;
	...
	}
		define dso_local void @foo(i32 %val){ entry:
		...
		%5 = load i32, i32* @foo.seed, align 4
		...
			%loadcast2 = bitcast i32* @foo.seed to i8*
			call void @variable_capture(i8* %loadcast2, i8* getelementptr inbounds ([7 x i8], [7 x i8]* @loadVar2, i32 0, i32 0), i64 %loadsizeofVar2)
				Bitcasted the address from *i32* to *i8** and assign to loadcast2. Captured Variable name *seed* from the second argument of *load* and assign %loadVar2. Compute the size from *load* instruction. Here it is i32, hence size is 4 and assign to *%loadsizeofVar2*.

Uninitialized global variable stored in BSS	char magic; /* globally uninitialized	... %6 = load i8, i8* @magic, align 1 ...	%loadcast3 = bitcast i8* @magic to i8* call void @variable_capture(i8* %loadcast3, i8* getelementptr inbounds ([7 x i8], [7 x i8]* @loadVar3, i32 0, i32 0), i64 %loadsizeofVar3)	Bitcasted the address from *i8* to *i8** and assign to loadcast3. Captured Variable name *magic* from the second argument of *load* and assign %loadVar3. Compute the size from *load* instruction. Here it is i8, hence size is 1 and assign to *%loadsizeofVar3*.
Initialized global variable stored in data segment	int cnt = 0; /* globally initialized	... %3 = load i32, i32* @cnt, align 4 ...	%loadcast4 = bitcast i8* @cnt to i8* call void @variable_capture(i8* %loadcast4, i8* getelementptr inbounds ([5 x i8], [5 x i8]* @loadVar4, i32 0, i32 0), i64 %loadsizeofVar4)	Bitcasted the address from *i32* to *i8** and assign to loadcast4. Captured Variable name *cnt* from the second argument of *load* and assign %loadVar4. Compute the size from *load* instruction. Here it is i32, hence size is 4 and assign to *%loadsizeofVar4*.
Initialized global variable stored in data segment in read-write	char pair [ ] = "5101", word [ ] = "etz", sym [ ] = "@#$&";	... %arrayidx50 = getelementptr inbounds [7 x i8], [7 x i8]* @word, i64 0, i64 %idxprom49 %22 = load i8, i8* %arrayidx50, align 1 ...	%loadcast5 = bitcast i8* %arrayidx50 to i8* call void @variable_capture(i8* %loadcast5, i8* getelementptr inbounds ([7 x i8], [7 x i8]* @loadVar5, i32 0, i32 0), i64 %loadsizeofVar5)	Bitcasted the address from *i8* to *i8** and assign to loadcast5. Since *load i*nstruction contains *arrayidx*, capture the variable name from the previous statement of *load* and assign to *loadVar5*. Compute the size from *load* instruction. Here it is i8, hence size is 1 and assign to *%loadsizeofVar5*.
Initialized global variable stored in data segment in read-only	const char key [] = "a2z";	... %arrayidx38 = getelementptr inbounds [4 x i8], [4 x i8]* @key, i64 0, i64 %idxprom37 %19 = load i8, i8* %arrayidx38, align 1 ...	%loadcast6 = bitcast i8* %arrayidx38 to i8* call void @variable_capture(i8* %loadcast6, i8* getelementptr inbounds ([4 x i8], [4 x i8]* @loadVar6, i32 0, i32 0), i64 %loadsizeofVar6, i32 10)	Bitcasted the address from *i8* to *i8** and assign to loadcast6. Since *load i*nstruction contains *arrayidx*, capture the variable name from the previous statement of *load* and assign to *loadVar6*. Compute the size from *load* instruction. Here it is i32, hence size is 4 and assign to *%loadsizeofVar6*.

Fig. 7. Sample Set of SQUIRREL Instrumentation Instructions for Classes of Memory Regions (Static)